

Size Balanced Tree

Chen Qifeng (Farmer John)

29 tháng 12 năm 2006

Nguồn gốc: Size Balanced Tree

IOI China candidate team papers / 2007 / day 1 / Chen Qifeng

Zhongshan Memorial Middle School, Guangdong, China

Email: 344368722@qq.com

Tóm tắt

Bài viết trình bày một chiến lược độc đáo để duy trì cân bằng trong cây tìm kiếm nhị phân thay đổi động, với hành vi kỳ vọng tối ưu ngay cả trong trường hợp xấu nhất. Đúng như tên gọi, **Size Balanced Tree** là cây tìm kiếm nhị phân được giữ cân bằng bằng kích thước. Nó đơn giản, hiệu quả và linh hoạt về mọi mặt. Cài đặt rất dễ, mô tả trực tiếp, đồng thời chứng minh tính đúng đắn và thời gian chạy cũng đơn giản một cách đáng ngạc nhiên. Thời gian chạy của nó ngang với những cây tìm kiếm nhị phân nhanh nhất đã biết. Hơn nữa, trong thực tế nó thường nhanh hơn nhiều cây tìm kiếm nhị phân cân bằng nổi tiếng khác vì có xu hướng tiến gần đến cây cân bằng hoàn hảo. Nó không chỉ hỗ trợ các thao tác cơ bản điển hình mà còn hỗ trợ SELECT và RANK.

Từ khóa. Size Balanced Tree; SBT; duy trì cân bằng.

Bài viết này được dành để tưởng nhớ Heavens.

1 Dẫn nhập

Trước khi trình bày Size Balanced Tree, cần nhắc lại cây tìm kiếm nhị phân và hai phép quay trên cây tìm kiếm nhị phân: quay trái và quay phải.

1.1 Cây tìm kiếm nhị phân

Cây tìm kiếm nhị phân là một loại cấu trúc dữ liệu nâng cao quan trọng. Nó hỗ trợ nhiều thao tác trên tập động, gồm SEARCH, MINIMUM, MAXIMUM, PREDECESSOR, SUCCESSOR, INSERT và DELETE. Nó có thể được dùng như một từ điển hoặc một hàng đợi ưu tiên.

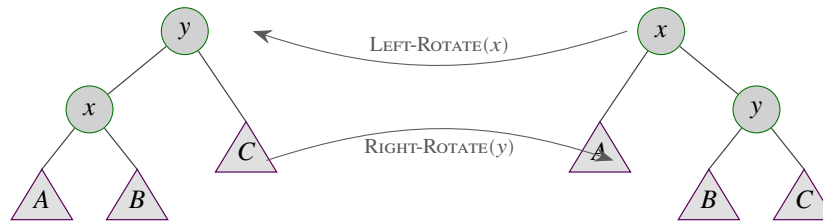
Một cây tìm kiếm nhị phân (viết tắt là BST) là một cây nhị phân có tổ chức. Mỗi nút trong BST có nhiều nhất hai con. Các khóa dùng để so sánh trong BST luôn được lưu sao cho thỏa tính chất cây tìm kiếm nhị phân:

Với một nút x trong cây tìm kiếm nhị phân, khóa của x không nhỏ hơn các khóa trong cây con trái và không lớn hơn các khóa trong cây con phải.

Với mỗi nút t , ta dùng các trường $left[t]$ và $right[t]$ để lưu hai con trỏ đến con của nó. Ta định nghĩa $key[t]$ là giá trị của nút t dùng cho so sánh. Ngoài ra, ta thêm $s[t]$, kích thước của cây con gốc t , để lưu số nút trong cây ấy. Đặc biệt, ta gọi 0 là con trỏ đến cây rỗng và đặt $s[0] = 0$.

1.2 Phép quay

Để giữ BST cân bằng, tức không suy biến thành một dãy xích, ta thường thay đổi cấu trúc con trỏ bằng các phép quay. Phép quay là thao tác cục bộ trong cây tìm kiếm, giữ nguyên tính chất cây tìm kiếm nhị phân nhưng thay đổi hình dạng cây.



Hình 1.1. Hai phép quay trái và quay phải là nghịch đảo của nhau.

1.2.1 Mã giả của quay phải

Phép quay phải giả thiết rằng con trái tồn tại.

```
Right-Rotate(t)
1 k <- left[t]
2 left[t] <- right[k]
3 right[k] <- t
4 s[k] <- s[t]
5 s[t] <- s[left[t]] + s[right[t]] + 1
6 t <- k
```

1.2.2 Mã giả của quay trái

Phép quay trái giả thiết rằng con phải tồn tại.

```
Left-Rotate(t)
1 k <- right[t]
2 right[t] <- left[k]
3 left[k] <- t
4 s[k] <- s[t]
5 s[t] <- s[left[t]] + s[right[t]] + 1
6 t <- k
```

2 Size Balanced Tree

Size Balanced Tree (viết tắt là SBT) là một loại cây tìm kiếm nhị phân được giữ cân bằng bằng kích thước. Nó hỗ trợ nhiều thao tác động cơ bản trong thời gian $\mathcal{O}(\log n)$:

INSERT(t, v) Chèn một nút có khóa v vào SBT gốc t .

DELETE(t, v) Xóa một nút có khóa v khỏi SBT gốc t . Nếu trong cây không có nút như vậy, xóa nút được tìm thấy cuối cùng.

FIND(t, v) Tìm nút có khóa v và trả về nút đó.

RANK(t, v) Trả về hạng của v trong cây gốc t . Nói cách khác, đây là một cộng với số khóa nhỏ hơn v trong cây ấy.

SELECT(t, k) Trả về nút đứng ở vị trí thứ k . Rõ ràng thao tác này bao gồm cả lấy phần tử nhỏ nhất và lớn nhất, vì GET-MIN tương đương SELECT($t, 1$), còn GET-MAX tương đương SELECT($t, s[t]$).

PRED(t, v) Trả về nút có khóa lớn nhất nhưng nhỏ hơn v .

SUCC(t, v) Trả về nút có khóa nhỏ nhất nhưng lớn hơn v .

Thông thường mỗi nút của SBT chứa khóa, con trái, con phải và một trường phụ nhưng hữu ích luôn được cập nhật: kích thước của nó, như đã định nghĩa ở phần trên. Cốt lõi của SBT gồm hai ràng buộc theo kích thước:

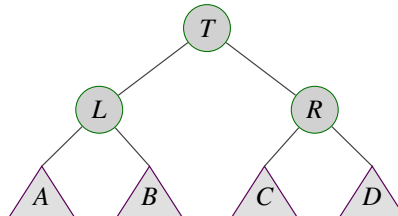
Với mọi nút được trỏ bởi t trong SBT, ta bảo đảm:

$$s[\text{right}[t]] \geq s[\text{left}[\text{left}[t]]], \quad s[\text{right}[t]] \geq s[\text{right}[\text{left}[t]]], \quad (\text{a})$$

$$s[\text{left}[t]] \geq s[\text{right}[\text{right}[t]]], \quad s[\text{left}[t]] \geq s[\text{left}[\text{right}[t]]]. \quad (\text{b})$$

Trong Hình 2.1, các nút L và R lần lượt là con trái và con phải của nút T . Các cây con A, B lần lượt là cây con trái và phải của L ; các cây con C, D lần lượt là cây con trái và phải của R . Tương ứng với hai tính chất (a) và (b), ta có

$$s[A], s[B] \leq s[R], \quad s[C], s[D] \leq s[L].$$



Hình 2.1. Ký hiệu các cây con quanh nút T .

3 Duy trì cân bằng

Giả sử ta cần chèn một nút có khóa v vào một BST. Thông thường ta dùng thủ tục sau:

```

Simple-Insert( $t, v$ )
1  If  $t = \emptyset$  then
2       $t \leftarrow \text{NEW-NODE}(v)$ 
3  Else
4       $s[t] \leftarrow s[t] + 1$ 
5      If  $v < \text{key}[t]$  then
6          Simple-Insert(left[ $t$ ],  $v$ )
7      Else
8          Simple-Insert(right[ $t$ ],  $v$ )

```

Sau khi thực hiện SIMPLE-INSERT, các tính chất (a) và (b) có thể không còn thỏa. Khi đó ta cần duy trì lại SBT.

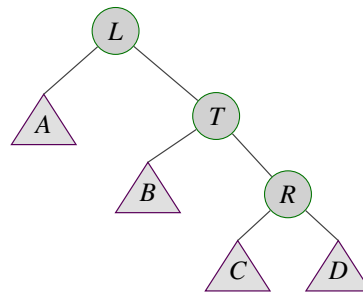
Thao tác then chốt trên SBT là một thủ tục duy nhất: MAINTAIN. MAINTAIN(t) được dùng để duy trì SBT gốc t . Trước khi gọi thủ tục này, giả thiết là các cây con của t đã là SBT.

Vì hai tính chất (a) và (b) đối xứng nhau, ta chỉ thảo luận chi tiết trường hợp ràng buộc (a) bị vi phạm.

Trường hợp 1: $s[\text{left}[\text{left}[t]]] > s[\text{right}[t]]$

Trong trường hợp $s[A] > s[R]$ tương ứng với hình 2.1 sau khi chèn vào $\text{left}[t]$, ta có thể sửa SBT bằng các bước sau:

1. Trước hết thực hiện RIGHT-ROTATE(T). Thao tác này biến cấu hình của hình 2.1 thành cấu hình của hình 3.1.

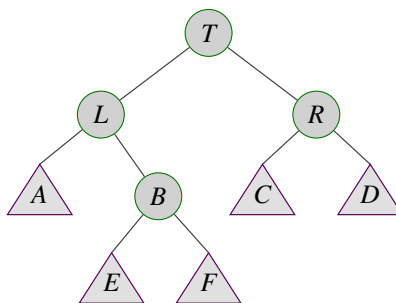


Hình 3.1. Kết quả sau RIGHT-ROTATE(T).

2. Đôi khi cây vẫn chưa là SBT, vì có thể xảy ra $s[C] > s[B]$ hoặc $s[D] > s[B]$. Do đó cần thực hiện MAINTAIN(T).
3. Sau đó, cấu hình cây con phải của nút L có thể đã thay đổi. Vì vẫn có khả năng vi phạm tính chất cân bằng, cần chạy lại MAINTAIN(L).

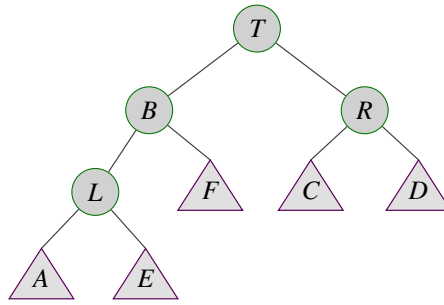
Trường hợp 2: $s[\text{right}[\text{left}[t]]] > s[\text{right}[t]]$

Trong trường hợp $s[B] > s[R]$ tương ứng với hình 3.2 sau khi chèn vào $\text{left}[t]$, việc duy trì phức tạp hơn trường hợp 1 một chút. Ta có thể sửa bằng các thao tác sau.



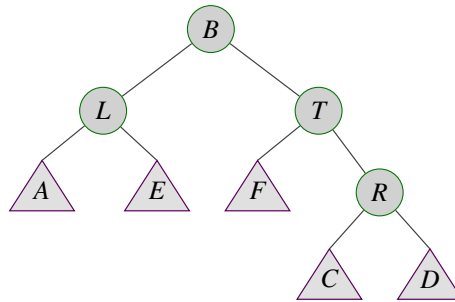
Hình 3.2. Trường hợp B là cây con phải của L.

1. Trước hết thực hiện LEFT-ROTATE(L). Sau thao tác này, cấu hình hình 3.2 được biến thành cấu hình hình 3.3.



Hình 3.3. Kết quả sau LEFT-ROTATE(L).

2. Tiếp theo thực hiện RIGHT-ROTATE(T). Thao tác này biến hình 3.3 thành hình 3.4.



Hình 3.4. Kết quả sau RIGHT-ROTATE(T).

3. Sau hai phép quay, cây trở nên khó đoán hơn. May mắn là trong hình 3.4, các cây con A, E, F, R vẫn là SBT. Vì vậy ta có thể thực hiện MAINTAIN(L) và MAINTAIN(T) để sửa hai cây con của nút B.
4. Sau bước trên, các cây con ấy đã là SBT. Nhưng tại nút B, các tính chất (a) và (b) vẫn có thể bị vi phạm. Do đó cần thực hiện lại MAINTAIN(B).

Trường hợp 3: $s[\text{right}[\text{right}[t]]] > s[\text{left}[t]]$

Trường hợp này đối xứng với trường hợp 1.

Trường hợp 4: $s[\text{left}[\text{right}[t]]] > s[\text{left}[t]]$

Trường hợp này đối xứng với trường hợp 2.

3.1 Mã giả của MAINTAIN chuẩn

Theo phân tích trên, ta dễ dàng cài đặt một thủ tục MAINTAIN thông thường.

```
Maintain(t)
1  If  $s[\text{left}[\text{left}[t]]] > s[\text{right}[t]]$  then
2      Right-Rotate(t)
3      Maintain(right[t])
4      Maintain(t)
5      Exit
6  If  $s[\text{right}[\text{left}[t]]] > s[\text{right}[t]]$  then
7      Left-Rotate(left[t])
8      Right-Rotate(t)
9      Maintain(left[t])
```

```

10     Maintain(right[t])
11     Maintain(t)
12     Exit
13 If s[right[right[t]]] > s[left[t]] then
14     Left-Rotate(t)
15     Maintain(left[t])
16     Maintain(t)
17     Exit
18 If s[left[right[t]]] > s[left[t]] then
19     Right-Rotate(right[t])
20     Left-Rotate(t)
21     Maintain(left[t])
22     Maintain(right[t])
23     Maintain(t)

```

3.2 Mã giả của MAINTAIN nhanh hơn và đơn giản hơn

Mã giả chuẩn hơi phức tạp và chậm. Thông thường ta có thể biết trước rằng tính chất (a) hoặc (b) đã thỏa. Vì vậy chỉ cần kiểm tra các trường hợp 1, 2 hoặc các trường hợp 3, 4 để tăng tốc. Khi đó ta thêm một biến Boolean *flag* để tránh các kiểm tra vô nghĩa. Nếu *flag* là *false*, ta kiểm tra trường hợp 1 và 2; ngược lại ta kiểm tra trường hợp 3 và 4.

```

Maintain(t, flag)
1  If flag = false then
2      If s[left[left[t]]] > s[right[t]] then
3          Right-Rotate(t)
4      Elseif s[right[left[t]]] > s[right[t]] then
5          Left-Rotate(left[t])
6          Right-Rotate(t)
7      Else exit
8  Elseif s[right[right[t]]] > s[left[t]] then
9      Left-Rotate(t)
10 Elseif s[left[right[t]]] > s[left[t]] then
11     Right-Rotate(right[t])
12     Left-Rotate(t)
13 Else exit
14 Maintain(left[t], false)
15 Maintain(right[t], true)
16 Maintain(t, false)
17 Maintain(t, true)

```

Vì sao có thể bỏ $\text{MAINTAIN}(\text{left}[t], \text{true})$ và $\text{MAINTAIN}(\text{right}[t], \text{false})$? Thời gian chạy của MAINTAIN là bao nhiêu? Câu trả lời nằm ở phần 6, phân tích.

4 Chèn

Thao tác chèn trên SBT rất đơn giản. Sau đây là mã giả của INSERT trên SBT.

4.1 Mã giả của INSERT

```
Insert(t, v)
1  If t = 0 then
2      t <- NEW-NODE(v)
3  Else
4      s[t] <- s[t] + 1
5      If v < key[t] then
6          Simple-Insert(left[t], v)
7      Else
8          Simple-Insert(right[t], v)
9      Maintain(t, v >= key[t])
```

5 Xóa

Tác giả mở rộng thao tác xóa cho tiện dùng. Nếu giá trị cần xóa không tồn tại trong SBT, ta xóa nút được tìm thấy cuối cùng và ghi lại nó. Sau đây là mã giả chuẩn của DELETE trên SBT.

5.1 Mã giả của DELETE chuẩn

```
Delete(t, v)
1  If s[t] <= 2 then
2      record <- key[t]
3      t <- left[t] + right[t]
4      Exit
5  s[t] <- s[t] - 1
6  If v = key[t] then
7      Delete(left[t], v[t] + 1)
8      key[t] <- record
9      Maintain(t, true)
10 Else
11     If v < key[t] then
12         Delete(left[t], v)
13     Else
14         Delete(right[t], v)
15     Maintain(t, v < key[t])
```

5.2 Mã giả của DELETE nhanh hơn và đơn giản hơn

Thực ra đây là thao tác xóa đơn giản nhất mà không cần hàm phụ khác. Ở đây $DELETE(t, v)$ là một hàm trả về giá trị bị xóa. Nó có thể làm hỏng cấu trúc SBT. Nhưng khi dùng cùng thao tác chèn ở trên, BST vẫn được giữ ở độ cao $\mathcal{O}(\log n^*)$, trong đó n^* là tổng số lần chèn, chứ không phải kích thước hiện tại.

```
Delete(t, v)
1  s[t] <- s[t] - 1
2  If (v = key[t]) or (v < key[t] and (left[t] = 0) or
    (v > key[t] and (right[t] = 0)) then
3      Delete <- key[t]
```

```

4      If (left[t] = 0) or (right[t] = 0) then
5          t <- left[t] + right[t]
6      Else
7          key[t] <- Delete(left[t], v[t] + 1)
8      Else
9          If v < key[t] then
10             Delete(left[t], v)
11          Else
12             Delete(right[t], v)

```

6 Phân tích

Rõ ràng MAINTAIN là một thủ tục đệ quy. Có thể bạn sẽ tự hỏi liệu nó có dừng hay không. Không cần lo: người ta đã chứng minh được rằng thời gian khấu hao của MAINTAIN chỉ là $\mathcal{O}(1)$.

6.1 Phân tích độ cao

Gọi $f[h]$ là số nút nhỏ nhất của một SBT có độ cao h . Ta có

$$f[h] = \begin{cases} 1, & h = 0, \\ 2, & h = 1, \\ f[h-1] + f[h-2] + 1, & h > 1. \end{cases}$$

6.1.1 Chứng minh

1. Dễ thấy $f[0] = 1$ và $f[1] = 2$.

2. Trước hết,

$$f[h] \geq f[h-1] + f[h-2] + 1 \quad (h > 1).$$

Với mỗi $h > 1$, giả sử t trở đến một SBT có độ cao h . Khi đó SBT này chứa một cây con có độ cao $h-1$. Không mất tính tổng quát, giả sử đó là cây con trái. Theo định nghĩa của $f[h]$, ta có $s[\text{left}[t]] \geq f[h-1]$. Trong cây con trái ấy có một cây con cao $h-2$, nói cách khác có một cây con kích thước ít nhất $f[h-2]$. Nhờ tính chất (b), ta có $s[\text{right}[t]] \geq f[h-2]$. Vì vậy

$$s[t] \geq f[h-1] + f[h-2] + 1.$$

Mặt khác,

$$f[h] \leq f[h-1] + f[h-2] + 1 \quad (h > 1).$$

Ta có thể xây dựng một SBT đúng $f[h]$ nút và có độ cao h . Gọi loại SBT này là $\text{tree}[h]$:

$$\text{tree}[h] = \begin{cases} \text{một BST có một nút,} & h = 0, \\ \text{một BST bất kỳ có hai nút,} & h = 1, \\ \text{một BST chứa } \text{tree}[h-1] \text{ và } \text{tree}[h-2] \text{ làm hai cây con,} & h > 1. \end{cases}$$

Tổng hợp hai phía trên, ta thu được $f[h] = f[h-1] + f[h-2] + 1$ với $h > 1$.

6.1.2 Độ cao xấu nhất

Thực ra $f[h]$ là hàm mũ. Giá trị chính xác của nó có thể tính từ truy hồi:

$$f[h] = \frac{\alpha^{h+3} - \beta^{h+3}}{\sqrt{5}} - 1 = \frac{\alpha^{h+3}}{\sqrt{5}} - 1 = \text{Fibonacci}[h+2] - 1 = \sum_{i=0}^h \text{Fibonacci}[i],$$

trong đó

$$\alpha = \frac{1 + \sqrt{5}}{2}, \quad \beta = \frac{1 - \sqrt{5}}{2}.$$

Ở đẳng thức giữa, thành phần theo β được bỏ qua theo cách xấp xỉ thường dùng trong bản gốc.

h	13	15	17	19	21	23	25	27	29	31
$f[h]$	986	2583	6764	17710	46367	121392	317810	832039	2178308	5702886

Bổ đề. Độ cao xấu nhất của một SBT có n nút là giá trị lớn nhất của h sao cho $f[h] \leq n$.

Giả sử $maxh$ là độ cao xấu nhất của một SBT kích thước n . Theo bổ đề trên,

$$\begin{aligned} f[maxh] &= \frac{\alpha^{maxh+3}}{\sqrt{5}} - 1 \leq n \\ \Rightarrow \frac{\alpha^{maxh+3}}{\sqrt{5}} &\leq n + 1.5 \\ \Rightarrow maxh &\leq \log_{\alpha}(\sqrt{5}(n + 1.5)) - 3 \\ \Rightarrow maxh &\leq 1.44 \log_2 n + 1.5 - 1.33. \end{aligned}$$

Vậy độ cao của SBT là $\mathcal{O}(\log n)$.

6.2 Phân tích MAINTAIN

Ta có thể chứng minh MAINTAIN hoạt động rất hiệu quả bằng những kết luận ở trên.

Có một giá trị rất quan trọng để đánh giá một BST tốt đến mức nào: độ sâu trung bình của mọi nút. Đây là thương của SD , tổng độ sâu của tất cả các nút, chia cho n . Nói chung giá trị này càng nhỏ thì BST càng tốt. Vì n cố định, ta muốn SD càng nhỏ càng tốt.

Bây giờ ta tập trung vào SD . Ý nghĩa của nó là khả năng chặn số lần MAINTAIN. Nhìn lại các điều kiện để thực hiện phép quay trong MAINTAIN, ta thấy điều đáng chú ý là sau các phép quay, SD luôn giảm.

Chẳng hạn trong trường hợp 1, so sánh hình 2.1 và hình 3.1, SD tăng thêm một giá trị âm:

$$s[right[t]] - s[left[left[t]]].$$

Tương tự, khi so sánh hình 3.2 với hình 3.4, SD tăng một giá trị nhỏ hơn -1 :

$$s[right[t]] - s[right[left[t]]] - 1.$$

Do độ cao là $\mathcal{O}(\log n)$, SD luôn được giữ ở mức $\mathcal{O}(n \log n)$. Sau một lần chen vào SBT, SD chỉ tăng $\mathcal{O}(\log n)$. Do đó

$$SD = n \times \mathcal{O}(\log n) - T = \mathcal{O}(\log n) \quad \Rightarrow \quad T = \mathcal{O}(n \log n),$$

trong đó T là số lần MAINTAIN có thực hiện phép quay. Tổng số lần MAINTAIN bằng T cộng với số lần MAINTAIN không quay. Vì số lần không quay là $\mathcal{O}(n \log n) + \mathcal{O}(T)$, thời gian khấu hao cho MAINTAIN là

$$\frac{\mathcal{O}(T) + \mathcal{O}(n \log n) + \mathcal{O}(T)}{n \log n} = \mathcal{O}(1).$$

6.3 Phân tích từng thao tác

Vì độ cao của SBT là $\mathcal{O}(\log n)$ và MAINTAIN là $\mathcal{O}(1)$ theo khâu hao, thời gian chạy của mọi thao tác cơ bản đều là $\mathcal{O}(\log n)$.

6.4 Phân tích DELETE nhanh hơn và đơn giản hơn

Gọi $P(n^*)$ là mệnh đề: một BST dùng thao tác DELETE nhanh hơn và đơn giản hơn, cùng với n^* lần INSERT chuẩn, có độ cao $\mathcal{O}(\log n^*)$. Ta chứng minh $P(n^*)$ đúng với mọi số nguyên dương n^* bằng quy nạp toán học.

6.4.1 Chứng minh

Ở đây tác giả chỉ đưa ra một chứng minh phác thảo.

Giả sử một nút t được kiểm tra bởi MAINTAIN($t, false$), ta có

$$\frac{s[left[t]] - 1}{2} \leq s[right[t]] \Rightarrow s[left[t]] \leq \frac{2s[t] - 1}{3}.$$

Vì vậy, nếu mọi nút trên đường từ một nút đến gốc đều được MAINTAIN kiểm tra, thì độ sâu của nút này là $\mathcal{O}(\log n)$.

1. Với $n^* = 1$, $P(n^*)$ hiển nhiên đúng.
2. Giả sử $P(n^*)$ đúng với $n^* < k$. Với $n^* = k$, sau những lần chen liên tiếp cuối cùng, các nút được MAINTAIN kiểm tra phải nối với nhau để tạo thành một cây. Với mỗi lá của cây này, cây con do nó trở tới không bị MAINTAIN thay đổi. Vì vậy độ sâu của các nút trong cây con này không lớn hơn

$$\mathcal{O}(\log n^*) + \mathcal{O}(\log n) = \mathcal{O}(\log n^*).$$

3. Do đó $P(n^*)$ đúng với $n^* = 1, 2, 3, \dots$

Theo cách này, thời gian khâu hao của MAINTAIN vẫn là $\mathcal{O}(1)$.

6.5 Phân tích MAINTAIN nhanh hơn và đơn giản hơn

Sau đây là thảo luận vì sao có thể bỏ MAINTAIN($left[t], true$) và MAINTAIN($right[t], false$).

Trong trường hợp 1 của hình 3.2, ta có

$$\begin{aligned} s[L] &\leq 2s[R] + 1 \\ \Rightarrow s[B] &\leq \frac{2s[L] - 1}{3} \leq \frac{4s[R] + 1}{3} \\ \Rightarrow s[E], s[F] &\leq \frac{2s[B] - 1}{3} \leq \frac{8s[R] + 3}{9} \\ \Rightarrow s[E], s[F] &\leq \left\lfloor \frac{8s[R] + 3}{9} \right\rfloor \leq s[R]. \end{aligned}$$

Do đó MAINTAIN($right[t], false$), tương ứng với MAINTAIN($T, false$) trong hình 3.1, có thể được bỏ. Còn MAINTAIN($left[t], true$) rõ ràng không cần thiết.

Trong trường hợp 2 của hình 3.2, ta có

$$s[A] \geq s[E], \quad s[F] \leq s[R].$$

Các bất đẳng thức này cũng có nghĩa là kích thước các cây con của E nhỏ hơn $s[A]$, và kích thước các cây con của F nhỏ hơn $s[R]$. Vì vậy MAINTAIN($right[t], false$) và MAINTAIN($left[t], true$) đều có thể được bỏ.

7 Ưu điểm

7.1 SBT chạy nhanh đến mức nào

7.1.1 Bài toán điển hình

Viết chương trình thực hiện n thao tác từ đầu vào:

1. Chèn một số cho trước vào tập.
2. Xóa một số cho trước khỏi tập.
3. Trả lời một số cho trước có nằm trong tập hay không.
4. Trả về hạng của một số cho trước trong tập.
5. Trả về số thứ k trong tập.
6. Trả về số đứng trước một số cho trước.
7. Trả về số kế tiếp một số cho trước.

7.1.2 Thống kê

Bảng dưới ghi lại ba biểu đồ thời gian chạy trong bản gốc. Đơn vị là giây.

Thử nghiệm	SBT	AVL	Treap	Randomized BST
Chèn 2 000 000 giá trị ngẫu nhiên	7.13	8.34	11.36	13.61
2 000 000 thao tác ngẫu nhiên: 66% chèn, 33% xóa	5.59	7.42	8.86	10.47
2 000 000 thao tác ngẫu nhiên: 20% chèn, 10% xóa, 60% hỏi	3.39	4.21	5.67	6.21

Trong thực tế, SBT hoạt động rất xuất sắc. Từ các biểu đồ trên, ta thấy SBT chạy nhanh hơn nhiều cây tìm kiếm nhị phân cân bằng khác khi dữ liệu là ngẫu nhiên. Hơn nữa, dữ liệu càng có thứ tự thì SBT càng chạy nhanh ngoài dự đoán. Chỉ mất 2 giây để chèn 2 000 000 nút có thứ tự vào một SBT.

7.2 SBT hoạt động hiệu quả đến mức nào

Độ sâu trung bình của tất cả các nút không tăng trong khi MAINTAIN chạy. Vì lý do này, SBT luôn có xu hướng trở thành một BST cân bằng hoàn hảo.

Chèn 2 000 000 nút với giá trị ngẫu nhiên						
Loại	SBT	AVL	Treap	Randomized BST	Splay	BST hoàn hảo
Độ sâu trung bình	19.2415	19.3285	26.5062	25.5303	37.1953	18.9514
Độ cao	24	24	50	53	78	20
Số lần quay	1568017	1395900	3993887	3997477	25151532	?

Chèn 2 000 000 nút với giá trị có thứ tự						
Loại	SBT	AVL	Treap	Randomized BST	Splay	BST hoàn hảo
Độ sâu trung bình	18.9514	18.9514	25.6528	26.2860	999999.5	18.9514
Độ cao	20	20	51	53	1999999	20
Số lần quay	1999979	1999979	1999985	1999991	0	?

7.3 Dễ gỡ lỗi

Ban đầu ta chỉ cần cài một BST đơn giản để bảo đảm phần chính của chương trình là đúng. Sau đó thêm MAINTAIN vào INSERT rồi gỡ lỗi. Nếu phát hiện lỗi, ta chỉ cần gỡ lỗi MAINTAIN. Hơn nữa, SBT không dựa trên ngẫu nhiên, nên việc gỡ lỗi ổn định hơn nhiều so với Treap, skip list, Randomized BST, v.v.

7.4 Đơn giản

SBT gần như giống hệt một BST đơn giản. Nếu bỏ thao tác MAINTAIN duy nhất trong INSERT, SBT trở lại thành BST thường. Bản thân MAINTAIN cũng khá đơn giản.

7.5 Gọn

Nhiều BST cân bằng như SBT, AVL, Treap, cây đỏ-đen, v.v. cần các trường phụ để giữ cân bằng. Nhưng nhiều trường trong số đó không hữu ích cho mục đích khác, chẳng hạn độ cao, hệ số ngẫu nhiên và màu. Ngược lại, SBT chứa một trường phụ hữu ích là kích thước. Nhờ có nó, ta có thể mở rộng BST để hỗ trợ chọn theo thứ tự và tính hạng.

7.6 Linh hoạt

Vì độ cao của SBT là $\mathcal{O}(\log n)$, ta có thể thực hiện SELECT trong $\mathcal{O}(\log n)$ ở trường hợp xấu nhất. Splay không hỗ trợ thao tác này tốt như vậy, vì độ cao có thể dễ dàng suy biến thành $\mathcal{O}(n)$, điều đã được thể hiện trong bảng trên.

Lời cảm ơn

Tác giả cảm ơn giáo viên tiếng Anh của mình, Fiona, vì sự giúp đỡ nhiệt tình.

Tài liệu tham khảo

1. G. M. Adelson-Velskii and E. M. Landis, “An algorithm for the Organization of Information”, *Soviet Math. Doklady*, 1962.
2. L. J. Guibas and R. Sedgewick, “A dichromatic Framework for Balanced Trees”, *Proceedings of the Nineteenth Annual IEEE Symposium on Foundations of Computer Science*, 1978.
3. D. D. Sleator and R. E. Tarjan, “Self-adjusting binary search trees”, *JACM*, 32, 652–686, 1985.
4. S. W. Bent and J. R. Driscoll, *Randomly balanced searchtrees*. Manuscript, 1991.
5. Raimund Seidel and Cecilia R. Aragon, *Randomized Search Trees*, 1996.
6. M. A. Weiss, *Data Structures and Algorithm Analysis in C++*, Third Edition, Addison Wesley, 2006.
7. R. E. Tarjan, “Sequential access in splay trees takes linear time”, *Combinatorica* 5(4), 367–378, 1985.
8. J. Nievergelt and E. M. Reingold, “Binary search trees of bounded balance”, *SIAM J. Computing*, 2, 33–43, 1973.

9. K. Mehlhorn and A. Tsakalidis, “Data Structures”, in *Handbook of Theoretical Computer Science*, Chapter 6, Vol. A, pp. 303–341, Elsevier Science Publishers, 1990.